UNIVERSIDAD GALILEO

TECNICO EN DESARROLLO DE SOFTWARE

SISTEMA DE GESTION DE INVENTARIO CON INTEGRACIÓN SQL Y NOSQL

Curso: Base de Datos

Catedrático: Mynor Ruiz

Integrantes:

25001309 Rafael Humberto Morales Lara

25003028 Kevin Estuardo Menéndez Lara

25004369 Erick Javier Cabrera Morales

25004942Vyanka Ivette Morales Gómez

Ciudad de Guatemala, marzo 2026

Contenido

Introducción....3
Objetivo del proyecto....3
Alcance....3
Descripción del sistema....3
Requerimientos....3
Requerimientos Funcionales....3
Requerimientos No Funcionales....5
Diseño del sistema....5
Diagrama Entidad-Relación....6
Explicación Relaciones....6
Diseño Lógico....7
Composición de las tablas....10
Implementación en MySQL Workbench....11
Creación base de datos:....11
Creación tabla rol:....11
Creación tabla empleado:....11
Creación tabla producto:....12
Creación tabla almacén:....12
Creación tabla inventario:....12
Creación tabla venta:....13
Creación tabla detalle_venta:....13
Implementación de Triggers en SQL....14
Creación de índices....19
Creación de Vistas y Procedimientos....20
Gestión de Usuarios y Seguridad....26
MongoDB....27
Seleccionamos la base de datos....27
Creamos las colecciones....27
Insertamos documentos de prueba....28
Realizamos consultas básicas....28
Justificación técnica....30
Conclusión....30

Introducción

Este proyecto presenta el diseño e implementación de un sistema de gestión de inventario para una empresa, incluyendo el control de productos y ventas. El objetivo principal es organizar y generar reportes que faciliten la toma de decisiones.

Objetivo del proyecto

Desarrollar un sistema que permita:

- Analizar los requerimientos del sistema
- Diseñar el modelo conceptual y lógico de las bases de datos.
- Transformarlo a modelo lógico
- Implementar el sistema utilizando tecnologías SQL y NoSQL
- Integrar mecanismos de consulta, modificación y generación de reportes.

Alcance

El sistema permite gestionar inventarios y ventas para un mejor manejo sobre el producto. No incluye funcionalidades de facturación, clientes ni implementaciones con sistemas externos. Este es un sistema para generar reportes y consultas sobre un producto y por lo tanto se enfoca en:

- Registro y consulta de productos.
- Control de stock por almacén.
- Generación de reportes de inventarios y ventas.
- Almacenamiento histórico en MongoDB.

Descripción del sistema

Las personas que utilizarán este sistema son empleados de la empresa, que desean consultar acerca del stock de uno o mas productos, divididos en administradores y operadores, los administradores podrán modificar, y hacer cambios sobre la información del producto, los almacenes, y consultar los empleados de la empresa, incluyendo eliminar usuarios de empleados que ya no se encuentran dentro de la empresa. Los operadores, solo podrán consultar la información de los productos, pero no podrán realizar modificaciones sin previa autorización de algún administrador. El sistema maneja información específica de los productos y las ventas que disminuyen el stock, para permitir un control sobre esta información.

Requerimientos

Requerimientos Funcionales

RF, 01: Registro de productos: El sistema debe permitir el registro la de nuevos productos en el inventario, incluyendo la siguiente información:

- Código del producto
- Nombre del producto
- Descripción
- Precio Unitario
- Stock disponible
- Ubicación dentro del almacén

RF-02: Actualización de productos: El sistema debe dar la posibilidad de modificar la información de los productos existentes, incluyendo cambio de nombre, descripción precio o ubicación.

RF-03: Eliminación de productos: El sistema debe posibilitar que se puedan eliminar productos del sistema siempre que no están vinculados a transacciones activas.

RF-04: Verificación de duplicidad: El sistema debe contener mecanismos para prevenir la duplicidad de productos, usando el código del producto como identificador único.

RF-05: Gestión de inventario: El sistema debe permitir consultar el inventario disponible de los productos, incluyendo el stock existen y su ubicación dentro de los diferentes almacenes.

RF-06: Reporte de inventario general: El sistema debe generar un reporte general de inventario que incluya:

- Código del producto
- Nombre del producto
- Stock actual
- Precio Unitario
- Valor total del inventario por producto (stock por precio)

RF-07: Reporte de productos por ubicación: El sistema debe permitir generar reportes de productos agrupados, ya sea por almacén o ubicación, para facilitar la organización del inventario.

RF-08: Reporte de ventas: El sistema debe generar reportes de transacciones de ventas, incluyendo la siguiente información como mínimo:

- Producto vendido
- Cantidad
- Fecha de transacción
- Valor total de la venta

RF-09: Búsquedas y filtros: El sistema debe permitir realizar consultas avanzadas, especialmente los siguientes filtros:

- Búsqueda de productos por rango de precios
- Identificación de productos con stock menor a un nivel determinado
- Búsqueda de productos por ubicación específica

RF-10: Gestión de usuarios: El sistema debe permitir la gestión de usuarios, incluyendo registro y administración de cuentas dentro del sistema.

RF-11: Gestión de roles: El sistema debe implementar roles de usuario para controlar el acceso al sistema, siendo estos:

- Administrador
- Operador

RF-12: Registro histórico en base de datos No relacional o NoSQL: El sistema debe de almacenar información histórica en una base de datos NoSQL, siendo incluida la siguiente información en cada registro:

- Historial de transacciones
- Cambios realizados en productos
- Comentarios registrados por los operadores

Requerimientos No Funcionales

RNF-01 Rendimiento: El sistema debe permitir consultas y generación de reportes de manera eficiente, inclusive con grandes volúmenes de datos.

RNF-02: Seguridad: El sistema debe implementar roles y permisos para restringir el acceso a determinadas operaciones según el tipo de usuario.

RNF-03: Escalabilidad: El sistema debe permitir la escalabilidad de la base de datos mediante técnicas de replicación y partición, garantizando disponibilidad y rendimiento.

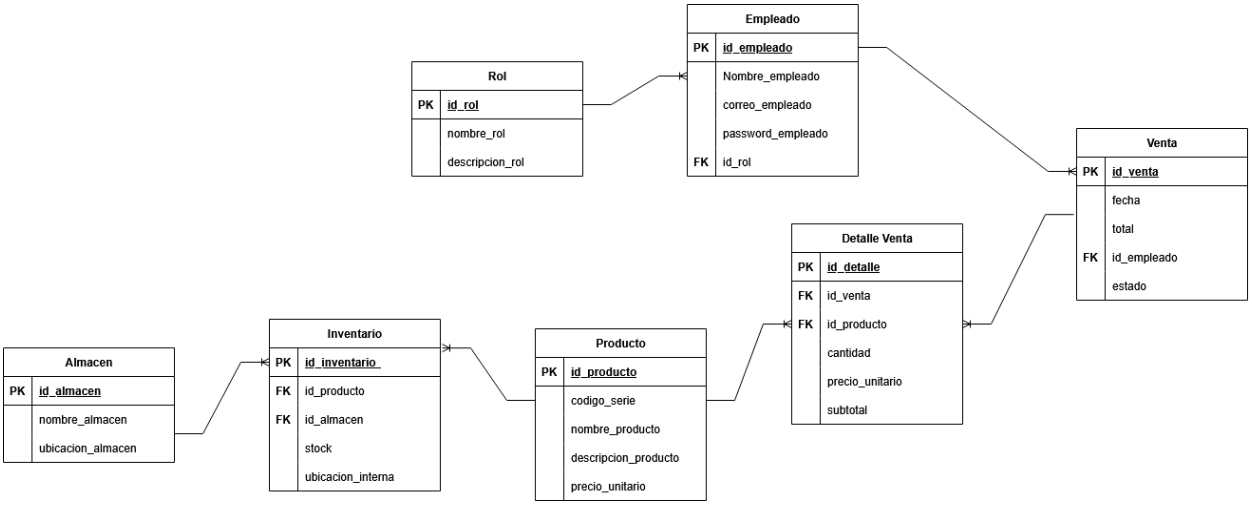
RNF-04: Integración de bases de datos: El sistema debe integrar bases de datos relaciones y no relacionales, dividiéndose en:

- MySQL para almacenamiento principal de datos.
- MongoDB para almacenamiento de registros históricos.

RNF-05: Disponibilidad: El sistema debe implementar replicación de datos para garantizar alta disponibilidad y tolerancia a fallos.

Diseño del sistema

Diagrama Entidad-Relación



Explicación Relaciones

Entidades:

- Roles
- Empleados
- Ventas
- Detalle Ventas
- Productos
- Inventario
- Almacenes

Un Rol tiene uno o muchos empleados – Cada empleado tiene asignado un rol específico.

Un Empleado puede realizar muchas ventas.

Un Producto puede ser vendido muchas veces.

Una venta tiene uno o más productos.

Un producto puede ser vendido varias veces.

Un almacén puede tener muchos productos

Un producto puede estar en muchos almacenes.

Diseño Lógico

Producto			
Campo	**Tipo de Dato**	**Restricciones**	**Descripción**
id_producto	Integer incremental	Primary Key	Identificador interno del producto
codigo_serie	Varchar(50)	Unique, not null	Código o serie del producto
Nombre_producto	Varchar(50)	Not null	Nombre del producto
Descripción_producto	Varchar(80)		Breve descripción del producto
Precio_unitario	Decimal (10,2)	Not null	Precio unitario del producto

Inventario			
Campo	**Tipo de Dato**	**Restricciones**	**Descripción**
Id_inventario	Integer	Primary Key	Identificador
Id_producto	Integer	Foreign Key – productos(id_producto)	Producto
Id_almacen	Integer	Foreign key – almacenes(id_almacen)	
Stock	Int		Cantidad disponible
Ubicacion_interna	Varchar(80)		Ubicacion dentro del almacen

Almacen			
Campo	**Tipo de Dato**	**Restricciones**	**Descripción**
Id_almacen	Int	Primary Key	Identificador interno del almacen
Nombre_almacen	Varchar(50)	Not null	Nombre del almacén
ubicacion_almacen	Varchar(80)	Not null	Dirección física del almacén

Detalle de Venta			
Campo	**Tipo de Dato**	**Restricciones**	**Descripción**
Id_detalle_venta	Integer	Primary Key	Identificador único del detalle de venta
Id_venta	Int	Foreign key – ventas(id_venta)	Venta a la que pertenece la transaccion
Id_producto	Int	Foreign key – productos(id_producto)	Identificador de product vendido
cantidad	Int	Not null	Cantidad de producto
Precio_unitario	Decimal(10,2)	Not null	Precio actual del producto
Subtotal	Decimal(10,2)	Not null	Cantidad por el precio unitario

Rol			
Campo	**Tipo de Dato**	**Restricciones**	**Descripción**
Id_rol	Int incremental	Primary Key	Identificador del rol
Nombre_rol	Varchar(50)	Unique	Nombre del rol
Descripción_rol	Varchar(100)		Explicación de las funciones del rol

Empleado			
Campo	**Tipo de Dato**	**Restricciones**	**Descripción**
Id_empleado	Int	Primary Key	Identificador de empleado
Nombre_empleado	Varchar(60)	Not null	Nombre del empleado
Correo_empleado	Varchar(60)	Unique	Correo del empleado
Password_empleado	Varchar(25)	Not null	Contraseña cifrada del empleado
Id_rol	int	Foreign Key-rol(id_rol)	Rol del empleado

Venta			
Campo	**Tipo de Dato**	**Restricciones**	**Descripción**
Id_ventas	Int	Primary Key	Identificador de la venta
Fecha	Datetime	Not null	Fecha de la venta
Total	Decimal(10,2)		Total, de la venta
id_empleado	Int	Foreign key – empleados(id_empleado)	Empleado que realice la venta
Estado	ENUM(‘pendiente’,’completada’cancelada’	DEFAULT(‘pendiente’)	Muestra el estado de la transacción

Composición de las tablas

Rol: id para identificar cada rol internamente en al BD, el rol debe tener nombre y una descripción de que es lo que hace.

Empleado: id para identificarlo internamente, el nombre del empleado, un correo, su contraseña cifrada y la id del rol asignado al empleado.

Venta: id para identificar la venta realizada, la fecha en la que se realiza la venta, el total de la venta y el id del empleado que realice la venta.

Producto: id de identificación interna, código de serie, SKU o algún similar que indique cosas como el modelo, serie, lote, etc. Nombre del producto, descripción breve del producto, el precio individual del producto.

Almacen: id de identificación interna del almacén, el nombre del almacén y su ubicación física.

Inventario: id de inventario para identificación, id del producto almacenado, id del almacén de donde se encuentra almacenado, stock, o cantidad disponible del producto y ubicación exacta del producto dentro del almacén.

Detalle Venta: id del detalle de la venta, id de la venta a la que está asociada, id del producto o los productos de la venta, cantidad individual de cada producto o del producto en la venta, precio unitario del o los productos y el subtotal de la venta.

Implementación en MySQL Workbench

Para implementar dentro de SQL Workbench, se utilizó el siguiente query para las tablas y base de datos:

Creación base de datos:

La base de datos debe ser creada para almacenar el inventario que va a contener el producto, la venta, el stock, para llevar el control que desea la empresa por sobre su producto.

```
create database sistema_inventario;
use sistema_inventario;
```

Creación tabla rol:

No todos los empleados tienen el nivel de autorización para realizar cambios dentro de la base de datos, para poder diferenciarlos, se crea una tabla de roles, ahí se normaliza que tipo de acceso y responsabilidad tienen dentro de la empresa, un operador puede consultar las ventas, el inventario, etc, pero un administrador deberia poder modificar información e inclusive tener control sobre quien puede consultar información mas sensible.

```
Create table rol
(
id_rol int auto_increment primary key,
nombre_rol varchar(50) not null,
descripcion_rol varchar(100)
);
```

Creación tabla empleado:

Una empresa puede tener una gran variedad de empleados, y una tabla para llevar control de sus roles, su nombre y su correo, permiten una mejor comunicación entre los empleados, además de evitar confusiones donde 2 empleados tengan nombres similares o casi idénticos, pero roles distintos.

```
Create table empleado
(
id_empleado int auto_increment primary key,
nombre_empleado varchar(60) not null,
correo_empleado varchar(60) not null unique,
password_empleado varchar(25),
id_rol int not null,
foreign key (id_rol) references rol(id_rol)
);
```

Creación tabla producto:

La tabla producto, se encarga de proveer la información acerca de un producto específico, mostrando la disponibilidad.

```
Create table producto
(
id_producto int auto_increment primary key,
codigo_serie varchar(50) not null unique,
nombre varchar(50) not null,
descripcion varchar(80),
precio decimal(10,2) not null
);
```

Creación tabla almacén:

Representa varios almacenes donde se almacenan los productos.

```
Create table almacen
(
id_almacen int auto_increment primary key,
nombre_almacen varchar(50) not null,
ubicacion_almacen varchar(80) not null
);
```

Creación tabla inventario:

Para registrar la cantidad de productos disponibles en cada almacen.

```
Create table inventario
(
id_inventario int auto_increment primary key,
id_producto int not null,
id_almacen int not null,
stock int not null,
ubicacion_ interna varchar(80),
foreign key (id_producto) references producto(id_producto),
foreign key (id_almacen)  references almacen(id_almacen)
);
```

Creación tabla venta:

La tabla ventas, muestra la transacción que realizan los empleados al vender los productos, identificando la fecha, el total de la venta y el empleado que hizo dicha venta, esto para llevar un control de comisiones por venta.

```
Create table venta
(
id_venta int auto_increment primary key,
fecha datetime not null,
total decimal(10,2) not null,
id_empleado int not null,
foreign key (id_empleado) references empleado(id_empleado)
);
```

Creación tabla detalle_venta:

La tabla de detalle_venta, almacena más detalladamente a la información de cada venta, permitiendo registrar varios productos por venta y referenciando al detalle general de la venta en específico.

La tabla detalle_venta, fue creada para representar la relación muchos a muchos entre ventas y productos según las reglas de normalización.

```
Create table detalle_venta
(
id_detalle int auto_increment primary key,
id_venta int not null,
id_producto int not null,
foreign key (id_venta) references venta(id_venta),
foreign key (id_producto) references producto(id_producto)
);
```

Implementación de Triggers en SQL

Durante el desarrollo del sistema, se llegó a la conclusión que era una necesidad controlar diferentes situaciones que pueden ocurrir durante el proceso de venta. Se determinaron tres estados posibles de una transacción, pendiente, completada y cancelada.

Para ser capaces de adaptarnos a este tipo de escenarios se realizó una modificación en la estructura de la tabla venta, agregando el atributo “estado”. Para ayudar a la clasificación de cada transacción según la situación dentro del sistema.

Se utilizó la siguiente instrucción:

```
alter table venta
ADD estado ENUM('pendiente','completada','cancelada')
DEFAULT 'pendiente';
```

Se utilizó el tipo ENUM, para restringir los valores posibles del estado.

- Pendiente: la venta ha sido registrada pero no confirmada.
- Completada: la venta se realizó exitosamente.
- Cancelada: la venta fue anulada.

Con el objetivo de mantener y garantizar la integridad de los datos y realizar automáticamente ciertas operaciones del sistema, se implementaron algunos triggers. Permitiendo validar la información antes de almacenarla y ejecutar automáticamente acciones determinadas cuando ocurren operaciones especificas dentro del sistema.

validar_producto_registro

Propósito: El objetivo de este trigger es el de:

- Validar que el precio del producto sea mayor que cero.
- El stock inicial no sea negativo.

Se ejecuta antes de insertar un nuevo producto en la tabla productos y evita que se registren productos con valores inválidos.

```
DROP TRIGGER IF EXISTS validar_producto_registro;

DELIMITER $$

CREATE TRIGGER validar_producto_registro
BEFORE INSERT ON producto
FOR EACH ROW
BEGIN

IF NEW.precio <= 0 THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'El precio debe ser mayor que cero';
END IF;

END$$

DELIMITER ;
```

validar_producto_update

Propósito: El objetivo de este trigger es el de:

- Asegurar que el precio del producto siga siendo válido.
- El stock no se vuelva negativo

Se ejecuta antes de actualizar un registro en la tabla productos y logra mantener la integridad de los datos cuando se realizan modificaciones en el sistema.

```
DROP TRIGGER IF EXISTS validar_producto_update;

DELIMITER $$

CREATE TRIGGER validar_producto_update
BEFORE UPDATE ON producto
FOR EACH ROW
BEGIN

IF NEW.precio <= 0 THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'El precio debe ser mayor que cero';
END IF;

END$$

DELIMITER ;
```

validar_stock_venta

Propósito: La función de este trigger es la de verificar que exista suficiente inventario antes de realizar una venta, cancelando la operación si la cantidad solicitada es mayor al stock disponible.

```
DROP TRIGGER IF EXISTS validar_stock_venta;

DELIMITER $$

CREATE TRIGGER validar_stock_venta
BEFORE INSERT ON detalle_venta
FOR EACH ROW
BEGIN

DECLARE stock_actual INT;

SELECT stock
INTO stock_actual
FROM inventario
WHERE id_producto = NEW.id_producto
LIMIT 1;

IF stock_actual <= 0 THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Stock insuficiente para realizar la venta';
END IF;

END$$

DELIMITER ;
```

actualizar_stock

Propósito: Este trigger tiene la función de descontar o reducir automáticamente la cantidad vendida del stock del producto vendido.

Se ejecuta después de registrar una venta dentro del sistema, permitiendo que el sistema mantenga el inventario actualizado sin intervención manual.

```
DELIMITER $$

CREATE TRIGGER actualizar_stock
AFTER INSERT ON detalle_venta
FOR EACH ROW
BEGIN

UPDATE inventario
SET stock = stock - 1
WHERE id_producto = NEW.id_producto
LIMIT 1;

END$$

DELIMITER ;
```

validar_estado

Propósito: Evitar inconsistencias del flujo, valida mediante un trigger e impide cambios de estado contrarios.

Si una venta fue cancelada, no se puede marcar como completada.

```
DROP TRIGGER IF EXISTS validar_estado;

DELIMITER $$

CREATE TRIGGER validar_estado
BEFORE UPDATE ON venta
FOR EACH ROW
BEGIN

IF OLD.estado = 'cancelada' AND NEW.estado = 'completada' THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'No se puede completar una venta cancelada';
END IF;

END$$

DELIMITER ;
```

devolver_stock_cancelacion

Propósito: Para mantener la consistencia del inventario cuando una venta previamente completada, pasa a cancelada. Los productos deben regresar automáticamente al inventario disponible.

Se ejecuta cuando el estado de una venta pasa de completada a cancelada.

```
DROP TRIGGER IF EXISTS devolver_stock_cancelacion;

DELIMITER $$

CREATE TRIGGER devolver_stock_cancelacion
AFTER UPDATE ON venta
FOR EACH ROW
BEGIN

IF OLD.estado = 'completada' AND NEW.estado = 'cancelada' THEN

UPDATE inventario i
JOIN (
select id_producto, COUNT(*) cantidad
FROM detalle_venta
WHERE id_venta = new.id_venta
GROUP BY id_producto
) dv ON i.id_producto = dv.id_producto
SET i.stock = i.tock + dv.cantidad;

END IF;
END$$

DELIMITER ;
```

Creación de índices

Con el fin de optimizar las consultas del sistema, fueron implementados índices en las columnas utilizadas frecuentemente. Como identificadores de productos, ventas y registros del inventario.

```
CREATE INDEX idx_detalle_venta_idventa ON detalle_venta(id_venta);

CREATE INDEX idx_detalle_venta_idproducto ON detalle_venta(id_producto);

CREATE INDEX idx_inventario_producto ON inventario(id_producto);

CREATE UNIQUE INDEX idx_producto_almacen ON inventario(id_producto,id_almacen);
```

Creación de Vistas y Procedimientos

```
DELIMITER $$

CREATE PROCEDURE sp_registrar_venta(
    IN p_id_empleado INT,
    IN p_id_producto INT,
    IN p_id_almacen INT,
    IN p_cantidad INT
)
BEGIN
    DECLARE v_precio DECIMAL(10,2) DEFAULT NULL;
    DECLARE v_stock INT DEFAULT NULL;
    DECLARE v_total DECIMAL(10,2) DEFAULT NULL;
    DECLARE v_id_venta INT DEFAULT NULL;
    DECLARE v_contador INT DEFAULT 1;
    DECLARE v_existe_empleado INT DEFAULT 0;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    IF p_cantidad <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'La cantidad debe ser mayor que cero';
    END IF;

    START TRANSACTION;

    SELECT COUNT(*)
    INTO v_existe_empleado
    FROM empleado
    WHERE id_empleado = p_id_empleado;

    IF v_existe_empleado = 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'El empleado no existe';
    END IF;

    SELECT precio
    INTO v_precio
    FROM producto
    WHERE id_producto = p_id_producto
    LIMIT 1;

    IF v_precio IS NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'El producto no existe';
    END IF;

    SELECT stock
    INTO v_stock
    FROM inventario
```

```
    WHERE id_producto = p_id_producto
      AND id_almacen = p_id_almacen
    LIMIT 1
    FOR UPDATE;

    IF v_stock IS NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'No existe inventario para ese producto en ese almacen';
    END IF;

    IF v_stock < p_cantidad THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Stock insuficiente';
    END IF;

    SET v_total = v_precio * p_cantidad;

    INSERT INTO venta (fecha, total, id_empleado)
    VALUES (NOW(), v_total, p_id_empleado);

    SET v_id_venta = LAST_INSERT_ID();

    WHILE v_contador <= p_cantidad DO
        INSERT INTO detalle_venta (id_venta, id_producto)
        VALUES (v_id_venta, p_id_producto);

        SET v_contador = v_contador + 1;
    END WHILE;

    UPDATE inventario
    SET stock = stock - p_cantidad
    WHERE id_producto = p_id_producto
      AND id_almacen = p_id_almacen;

    COMMIT;
END $$

DELIMITER ;
```

Procedimiento almacenado

```
SHOW PROCEDURE STATUS
WHERE Db = 'sistema_inventario';
```

Result Grid

Db	Name	Type	Definer	Modified	Created
sistema_inventario	sp_cargar_catalogos_base	PROCEDURE	root@localhost	2026-03-09 21:06:28	2026-03-09 21:06:28
sistema_inventario	sp_cargar_datos_prueba	PROCEDURE	root@localhost	2026-03-08 22:46:02	2026-03-08 22:46:02
sistema_inventario	sp_cargar_usuarios_empleados	PROCEDURE	root@localhost	2026-03-09 21:06:28	2026-03-09 21:06:28
sistema_inventario	sp_generar_ventas_random	PROCEDURE	root@localhost	2026-03-09 21:06:28	2026-03-09 21:06:28
sistema_inventario	sp_registrar_venta	PROCEDURE	root@localhost	2026-03-09 20:59:35	2026-03-09 20:59:35

Ejecución del procedimiento

```
CALL sp_registrar_venta(1, 1, 1, 1, 2);
SELECT *
FROM venta
ORDER BY id_venta DESC
LIMIT 5;
```

Result Grid

id_venta	fecha	total	id_empleado	id_usuario
905	2026-03-09 21:18:38	357.96	1	1
904	2026-03-09 21:18:30	357.96	1	1
903	2026-03-09 21:17:29	357.96	1	1
902	2026-03-09 21:16:34	357.96	1	1
901	2026-03-09 21:16:30	357.96	1	1
NULL	NULL	NULL	NULL	NULL

Vistas

```
SHOW FULL TABLES IN sistema_inventario
WHERE Table_type = 'VIEW';
```

Tables_in_sistema_inventario	Table_type
vw_productos_por_ubicacion	VIEW
vw_reporte_inventario_general	VIEW
vw_reporte_ventas_simuladas	VIEW

Vista de inventario general

```
SELECT *
FROM vw_reporte_inventario_general
LIMIT 20;
```

id_producto	codigo_serie	nombre_producto	stock_actual	precio_unitario	valor_total
1	P0001	Producto 1	314	178.98	56199.72
2	P0002	Producto 2	387	576.59	223140.33
3	P0003	Producto 3	244	371.02	90528.88
4	P0004	Producto 4	311	100.31	31196.41
5	P0005	Producto 5	330	338.49	111701.70
6	P0006	Producto 6	274	391.51	107273.74
7	P0007	Producto 7	224	917.09	205428.16
8	P0008	Producto 8	336	460.92	154869.12

vw_reporte_inventario_general 9

Read Only

Vista de productos por ubicación

```
SELECT *
FROM vw_productos_por_ubicacion
LIMIT 20;
```

id_almacen	nombre_almacen	ubicacion_almacen	ubicacion_interna	id_producto	codigo_serie	nombre_producto	sto
1	Almacen Central	Zona 1	Pasillo 1 - Estante 10	1	P0001	Producto 1	49
1	Almacen Central	Zona 1	Pasillo 1 - Estante 9	2	P0002	Producto 2	56
1	Almacen Central	Zona 1	Pasillo 1 - Estante 2	3	P0003	Producto 3	60
1	Almacen Central	Zona 1	Pasillo 1 - Estante 9	4	P0004	Producto 4	74
1	Almacen Central	Zona 1	Pasillo 1 - Estante 6	5	P0005	Producto 5	40
1	Almacen Central	Zona 1	Pasillo 1 - Estante 1	6	P0006	Producto 6	21
1	Almacen Central	Zona 1	Pasillo 1 - Estante 5	7	P0007	Producto 7	37

Ventas simuladas

```
SELECT *
FROM vw_reporte_ventas_simuladas
LIMIT 20;
```

id_venta	fecha	id_usuario	nombre_usuario	id_empleado	nombre_empleado	id_producto	codigo_serie
303	2025-09-12 02:14:27	25	Usuario 25	1	Empleado 1	17	P0017
303	2025-09-12 02:14:27	25	Usuario 25	1	Empleado 1	13	P0013
303	2025-09-12 02:14:27	25	Usuario 25	1	Empleado 1	14	P0014
304	2026-03-06 11:00:27	84	Usuario 84	1	Empleado 1	2	P0002
304	2026-03-06 11:00:27	84	Usuario 84	1	Empleado 1	17	P0017
310	2025-11-21 10:53:27	17	Usuario 17	1	Empleado 1	13	P0013
310	2025-11-21 10:53:27	17	Usuario 17	1	Empleado 1	20	P0020

vw_reporte_ventas_simuladas 11

Read Only

Muestra de que hay usuarios

```sql
SELECT *
FROM venta
ORDER BY id_venta DESC
LIMIT 10;
```

id_venta	fecha	total	id_empleado	id_usuario
905	2026-03-09 21:18:38	357.96	1	1
904	2026-03-09 21:18:30	357.96	1	1
903	2026-03-09 21:17:29	357.96	1	1
902	2026-03-09 21:16:34	357.96	1	1
901	2026-03-09 21:16:30	357.96	1	1
900	2026-02-18 23:22:37	2047.53	2	15
899	2025-12-02 08:47:37	503.33	6	43
898	2025-12-12 07:48:37	908.70	1	35

Total de Usuarios

```sql
SELECT COUNT(*) AS total_usuarios
FROM usuario;
```

total_usuarios
100

Gestión de Usuarios y Seguridad

Para implementar un mecanismo de control de acceso, se utilizó MySQL para definir dos roles principales:

- Administrador : Usuario con permisos completos sobre la base de datos.
- Operador : Usuario con permisos limitados, enfocado en registrar ventas y consultar información del inventario.

```
create role administrador;
create role operador;
```

Se concedieron los permisos correspondientes a cada rol:

```
Grant all privileges
on sistema_inventario.* to administrador;
```

```
grant select on sistema_inventario.producto to operador;
grant select on sistema_inventario.inventario to operador;
grant select on sistema_inventario.inventario to operador;
grant insert on sistema_inventario.venta to operador;
grant insert on sistema_inventario.detalle_venta to operador;
```

MongoDB

Se implementaron tres colecciones:

- HistorialTransacciones
- CambiosProductos
- ComentariosOperadores

Estas colecciones permiten almacenar información histórica no estructurada, útil para aditoria y análisis.

Seleccionamos la base de datos

Nos indica que trabajaremos en la base de datos “sistema inventario”

```
// Seleccionar la base de datos
use sistema_inventario
```

Creamos las colecciones

db.createCollection(“transacciones_historicas”): Crea la colección donde se guardan las transacciones realizadas en el inventario.

db.createColection(“hostorial_modificaciones”): Crea al colección para registrar cambios o ajustes hechos a los productos.

db.createCollection(“comentarios_operadores”): Crea la colección para almacenar observaciones y comentarios de los operadores.

```
// Seleccionar la base de datos
use sistema_inventario

// Crear colecciones
db.createCollection("transacciones_historicas")
db.createCollection("historial_modificaciones")
db.createCollection("comentarios_operadores")
```

Insertamos documentos de prueba

db.transacciones_historicas.insertOne: Inserta una transacción histórica con código de producto, cantidad, fecha y valor total.

db.historial_modificaciones.insertOne: Inserta un registro de modificación indicando que producto se cambió, que ajuste se hizo, cuando y quien lo realizó.

db.comentarios_operadores.insertOne: Inserta un comentario de un operador sobre un producto en específico.

```
// Insertar documentos de prueba en cada colección
db.transacciones_historicas.insertOne({
  codigo_producto: "P001",
  cantidad: 10,
  fecha: ISODate("2025-12-15"),
  valor_total: 500.00
})

db.historial_modificaciones.insertOne({
  codigo_producto: "P002",
  cambio: "Stock ajustado",
  fecha: ISODate("2026-01-10"),
  usuario: "operador1"
})

db.comentarios_operadores.insertOne({
  codigo_producto: "P003",
  comentario: "Producto con baja rotación",
  fecha: ISODate("2026-02-20"),
  operador: "María"
})
```

Realizamos consultas básicas

db.transacciones_historicas.find: Muestra todos los documentos guardados en la colección de transacciones históricas.

db.transacciones_historicas.aggregate: Agrupa las transacciones por producto y calcula el total vendido y el valor acumulado.

db.comentarios_operadores.find: Filtra los comentarios para mostrar únicamente los realizados por la operadora “Maria”

```mongo
// Consultas basicas (querys)
db.transacciones_historicas.find({})   // Ver todas las transacciones

db.transacciones_historicas.aggregate([
  { $group: {
      _id: "$codigo_producto",
      total_vendido: { $sum: "$cantidad" },
      valor_total: { $sum: "$valor_total" }
  }}
])   // Agrupar ventas por producto

db.comentarios_operadores.find({ operador: "Maria" })   // Ver comentarios de un operador especifico
```

Justificación técnica

- MySQL: Maneja datos estructurados y relaciones entre entidades.
- MongoDB: Almacena datos históricos flexibles y no estructurados.
- Integración: Ambos motores se complementan para cubrir necesidades operativas y analíticas.

Conclusión

El sistema desarrollado permite una gestión eficiente del inventario, combinando tecnologías SQL y NoSQL. La arquitectura garantiza escalabilidad, seguridad y trazabilidad, cumpliendo con los requerimientos del curso simulando un entorno empresarial real.